Random bytes in the bridge: four approaches

SpqrBridge slides, randomness alternatives

SpqrBridge

November 2026

Aeneas code is deterministic. VCVio proofs need distributions. Where does randomness enter?

The running example: `mask`

Rust `mask`

```
fn mask(rng: &mut R, v: u8)
  let mut pad = [0u8; 1];
  rng.fill_bytes(&mut pad);
  (pad[0], v ^ pad[0])
```

`pad` = what the generator writes
one fixed answer

input `v`

get the byte `pad`

same output

VCVio `Crypto.mask`

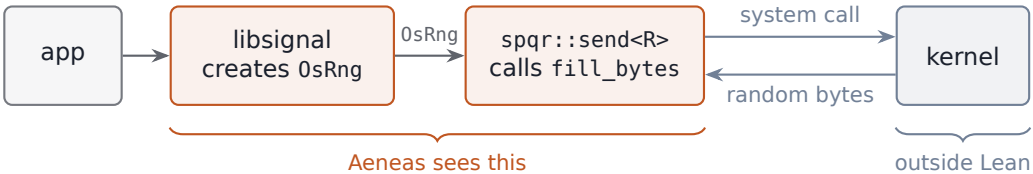
```
def mask (v : U8) :=
  let pad ← $ᵗ U8
  return (pad, v ⊕ pad)
```

`pad` = uniform random byte
a distribution over 256 answers

Same computation. Only the line that gets `pad` differs.

`Crypto.mask` is what `mask` computes when the generator is perfect.

Where the bytes really come from



Inside `send`, each `fill_bytes` on `OsRng` is a system call.

`OsRng` holds no state. Every approach trusts: “the kernel’s bytes are uniform”.

The problem



Goal: *if* the generator gives uniform bytes, *then* Rust `mask` = `Crypto.mask`.

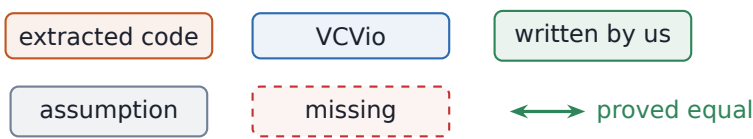
Next, we go through several approaches that differ in how they state that “if”.

How we compare them

Q1. Is the “if” a Lean statement, in one theorem?

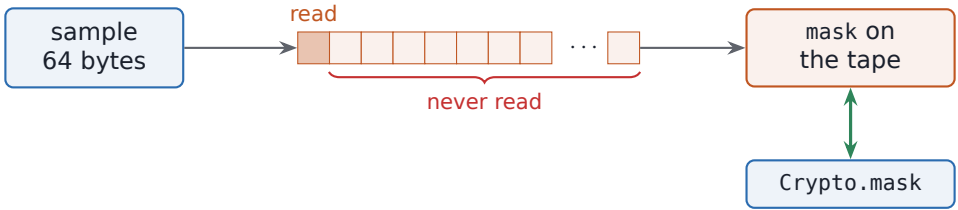
Q2. How much extra work per function?

How to read the diagrams



1. Random tape

spqr-sm-bridge before #10. Replaced.



Roll all the dice first. The code reads them.

For

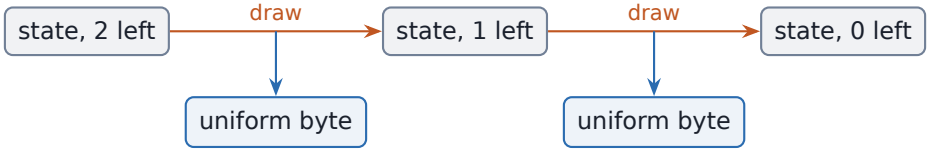
+ No change to Aeneas
+ Simple

Against

– Q1: no. The “if” is prose
– Q2: a tape length per function
– Q2: lemmas for unread bytes

2. Contract on the generator state

bridging-examples `main`. Removed in #1.



Assume what the state looks like before each draw (`RngContract`).

For

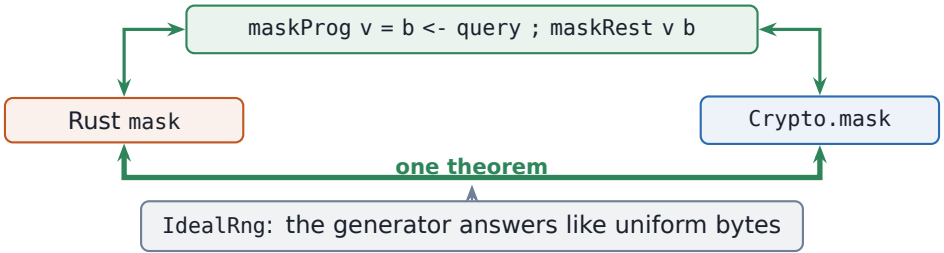
+ No change to Aeneas
+ Q1: yes

Against

- Q2: a draw count per function
- Randomness only from the state. `OsRng` keeps it in the OS

3. Oracle program[1] + `IdealRng`

bridging-examples#3, spqr-sm-bridge#10



`maskRest v b`: the unchanged extracted `mask`, with a generator that returns `b`.

So the bytes are an argument (`sendRest` in #10).

For

+ No change to Aeneas
+ Q1: yes
+ `IdealRng` is not empty: a uniform tape meets it

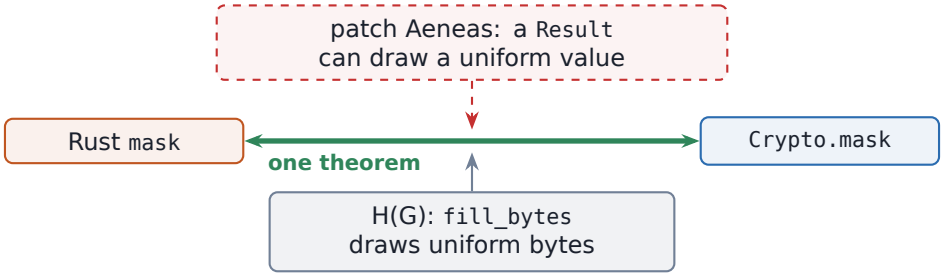
Against

- Q2: a program per function
- A budget, one state per program
- Randomness must come from the state. `OsRng` keeps it in the OS

[1] toy-bridge: the oracle idea behind #10 and #3, with the function copied by hand and without `IdealRng`.

4. Sampling effect in `Result`

Sergiu, proposal



Roll the dice inside the code, at the call.

For

+ Q1: yes
+ Q2: no extra program
+ Fits a stateless `OsRng`

Against

- Changes `Result` for all users
- Functional proofs stop at a draw
- First patch withdrawn, new one not seen

3 and 4: same trust, different shape

3 `IdealRng`

all answers of a program
look uniform

global, with a budget
Aeneas unchanged

4 H(G)

each call
draws uniform bytes

local, no budget
Aeneas patched

In both, “`OsRng` meets the assumption” stays outside Lean.

Side by side

	Aeneas unchanged	Q1: “if” in Lean	Q2: extra work
1 (Tape)	**yes**	**no**	tape length
2 (Contract)	**yes**	**yes**	draw counts
3 (Oracle)	**yes**	**yes**	program per function
4 (Effect)	**no**	**yes**	none